R_Markdown_Valerie

Valerie Yu

2026-03-13

Valerie's R Assignment

Setting up the R Markdown and loading in R library.

Part I

Data Inspection

`fang_et_al_genotypes.txt`

- There are 2782 rows and 986 columns.
- The first 3 columns are `Sample_ID`, `JG_OTU`, and `Group`. Then the rest are SNP data (983).

`snp_position.txt`

- There are 983 rows and 15 columns.
- The columns are named `SNP_ID`, `Chromosome`, `Position`, etc.

```
# Loading in data from "data" folder and looking at column names.
genotypes <- read_tsv("data/fang_et_al_genotypes.txt")
```

```
## Rows: 2782 Columns: 986
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr (986): Sample_ID, JG_OTU, Group, abph1.20, abph1.22, ae1.3, ae1.4, ae1.5...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
snp_position <- read_tsv("data/snp_position.txt")
```

```
## Rows: 983 Columns: 15
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr (9): SNP_ID, Chromosome, Position, alt_pos, mult_positions, amplicon, cd...
## dbl (6): cdv_marker_id, Genaissance_daa_id, Sequenom_daa_id, count_amplicons...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
# Inspecting the data
dim(genotypes) # 2782 rows  986 columns
```

```
## [1] 2782  986
```

```r
colnames(genotypes)[1:15] # variable names
```

```
##  [1] "Sample_ID" "JG_OTU"    "Group"     "abph1.20"  "abph1.22"  "ae1.3"
##  [7] "ae1.4"     "ae1.5"     "an1.4"     "ba1.6"     "ba1.9"     "bt2.5"
## [13] "bt2.7"     "bt2.8"     "Fea2.1"
```

```r
# head(genotypes) # 1st Sample_ID, 3rd Group
table(genotypes$Group)
```

```
##
## TRIPS ZDIPL ZLUXR ZMHUE ZMMIL ZMMLR ZMMMR ZMPBA ZMPIL ZMPJA ZMXCH ZMXCP ZMXIL
##    22    15    17    10   290  1256    27   900    41    34    75    69     6
## ZMXNO ZMXNT ZPERR
##     7     4     9
```

```r
  # maize count: ZMMIL 290, ZMMLR 1256, and ZMMMR 27 = 1573
  # teosinte count: ZMPBA 900, ZMPIL 41, and ZMPJA 34 = 975


dim(snp_position) # 983 rows  15 columns
```

```
## [1] 983  15
```

```r
colnames(snp_position)[1:15] # variable names
```

```
##  [1] "SNP_ID"               "cdv_marker_id"        "Chromosome"
##  [4] "Position"             "alt_pos"              "mult_positions"
##  [7] "amplicon"             "cdv_map_feature.name" "gene"
## [10] "candidate/random"     "Genaissance_daa_id"   "Sequenom_daa_id"
## [13] "count_amplicons"      "count_cmf"            "count_gene"
```

```r
# head(snp_position) # 1st SNP_ID, 3rd Chromosome, 4th Position, etc.
table(snp_position$Chromosome) # 6 multiple and 27 unknown
```

```
##
##        1       10        2        3        4        5        6        7
##      155       53      127      107       91      122       76       97
##        8        9 multiple  unknown
##       62       60        6       27
```

Data Processing

`fang_et_al_genotypes.txt` First, I am cutting out the groups I need for maize and teosinte.

```r
# Filtering `fang_et_al_genotypes.txt`
maize_genotypes <- genotypes %>% filter(Group == "ZMMIL" |
                                        Group == "ZMMLR" |
                                        Group == "ZMMMR" )
teosinte_genotypes <- genotypes %>% filter(Group == "ZMPBA" |
                                           Group == "ZMPIL" |
                                           Group == "ZMPJA" )
# Checking: both result is 3, which is accurate
n_distinct(maize_genotypes$Group)
```

```
## [1] 3
```

```r
n_distinct(teosinte_genotypes$Group)
```

```
## [1] 3
```

Then I will transpose the genotype files so that columns become rows by using t() before joining.

```r
# MAIZE
t_maize_genotypes <- as.data.frame(t(maize_genotypes))
t_maize_genotypes <- t_maize_genotypes[-c(1,2,3),]
dim(t_maize_genotypes) # 983 rows just like snp_position
```

```
## [1]  983 1573
```

```r
# Add Sample_ID back
colnames(t_maize_genotypes) <- maize_genotypes$Sample_ID

# TEOSINTE
t_teosinte_genotypes <- as.data.frame(t(teosinte_genotypes))
t_teosinte_genotypes <- t_teosinte_genotypes[-c(1,2,3),]
dim(t_teosinte_genotypes) # 983 rows just like snp_position
```

```
## [1] 983 975
```

```r
# Add Sample_ID back
colnames(t_teosinte_genotypes) <- teosinte_genotypes$Sample_ID
```

Now I am cleaning snp_position and fixing the join key for genotype files. Assignment says files formated with 1st column is "SNP_ID", the 2nd column is "Chromosome", the 3rd column is "Position", and subsequent columns are genotype data from either maize or teosinte individuals." help: `?rownames_to_column` and _https://stackoverflow.com/questions/29511215/how-can-i-convert-row-names-into-the-first-column_

```r
# Clean snp_position. We only want column 1, 3, 4
cleaned_snp_position <- select(snp_position,
                               "SNP_ID", "Chromosome", "Position")
# I don't have SNP_ID column name from genotype data so here I add it.
t_maize_genotypes <- tibble::rownames_to_column(t_maize_genotypes, "SNP_ID")
t_teosinte_genotypes <- tibble::rownames_to_column(t_teosinte_genotypes, "SNP_ID")
```

Joining snp_position to both of these genotype files. help: `?left_join` and _https://r4ds.hadley.nz/joins.html_

```
# MAIZE
joined_maize <- left_join(
  x = t_maize_genotypes,
  y = cleaned_snp_position,
  by = "SNP_ID")
tail(names(joined_maize), 5) # it stuck it onto the end, lets move it to the front
```

```
## [1] "CU0201"     "S0215"      "CU0202"     "Chromosome" "Position"
```

```
joined_maize <- joined_maize %>% select(SNP_ID, Chromosome, Position, everything())


# TEOSINTE
joined_teosinte <- left_join(
  x = t_teosinte_genotypes,
  y = cleaned_snp_position,
  by = "SNP_ID")
tail(names(joined_teosinte), 5) # it stuck it onto the end, lets move it to the front
```

```
## [1] "TIP-272A"   "TIP-356-A"  "TIP-512"    "Chromosome" "Position"
```

```
joined_teosinte <- joined_teosinte %>% select(SNP_ID, Chromosome, Position, everything())
```

Now let's check the new datasets : `joined_maize` and `joined_teosinte`

```
unique(joined_maize$Chromosome) # we need to remove multiple and unknown
```

```
##  [1] "2"        "5"        "1"        "3"        "4"        "6"
##  [7] "9"        "8"        "multiple" "7"        "unknown"  "10"
```

```
joined_maize <- filter(joined_maize,
                Chromosome != "multiple" & Chromosome != "unknown")
unique(joined_maize$Chromosome) # gone!
```

```
##  [1] "2"  "5"  "1"  "3"  "4"  "6"  "9"  "8"  "7"  "10"
```

```
unique(joined_teosinte$Chromosome) # we need to remove multiple and unknown
```

```
##  [1] "2"        "5"        "1"        "3"        "4"        "6"
##  [7] "9"        "8"        "multiple" "7"        "unknown"  "10"
```

```
joined_teosinte <- filter(joined_teosinte,
                Chromosome != "multiple" & Chromosome != "unknown")
unique(joined_teosinte$Chromosome)  # gone!
```

```
##  [1] "2"  "5"  "1"  "3"  "4"  "6"  "9"  "8"  "7"  "10"
```

Fixing position and removing positions that are not numeric, and hence can not be sorted by increasing position values.

```
class(joined_maize$Position) # character, should be numeric so I don't sort wrong
```

```
## [1] "character"
```

```
cleaned_maize <- joined_maize %>%
  mutate(Position = as.numeric(Position)) %>%
  filter(!is.na(Position))
```

```
## Warning: There was 1 warning in `mutate()`.
## i In argument: `Position = as.numeric(Position)`.
## Caused by warning:
## ! NAs introduced by coercion
```

```
cleaned_teosinte <- joined_teosinte %>%
  mutate(Position = as.numeric(Position)) %>%
  filter(!is.na(Position))
```

```
## Warning: There was 1 warning in `mutate()`.
## i In argument: `Position = as.numeric(Position)`.
## Caused by warning:
## ! NAs introduced by coercion
```

This is how I would generate the final files for 1 chromosome.

```
dim(cleaned_maize) # 1576 columns
```

```
## [1]  939 1576
```

```
test_chr_1 <- cleaned_maize %>%
  filter(Chromosome == "1")
test_chr_1_increase <- test_chr_1 %>%
  arrange(Position) %>%
  mutate(across(4:1576, function(x){
    ifelse(x == "?/?", "?", x)
  })
  )

test_chr_1_decrease <- test_chr_1 %>%
  arrange(desc(Position))%>%
  mutate(across(4:1576, function(x){
    ifelse(x == "?/?", "-", x)
  })
  )
```

Creating a function.

```r
final_files <- function (df, chr, name){

  chr_df <- df %>% filter(Chromosome == chr)
  chr_increase <- chr_df %>%
    arrange(Position) %>%
    mutate(across(-(1:3), function(x){
    ifelse(x == "?/?", "?", x)
  })
  )

  chr_decrease <- chr_df %>%
  arrange(desc(Position))%>%
  mutate(across(-(1:3), function(x){
    ifelse(x == "?/?", "-", x)
  })
  )
# write_csv(dataframe, output path should be final_results, delimiters)
# I need no spaces with it in the final_maize or final_teosinte folder which is inside the final_result
# set up should be : final_results/final_maize/maize_chr1_increasing.csv
  # which means that the actual file names requires
    # name + _chr + "chr" (the actual value) + _increasg.csv
   write_csv(chr_increase, paste0("final_results/final_", name, "/",
                                  name, "_chr", chr,
                                  "_increasing.csv") )
   write_csv(chr_decrease, paste0("final_results/final_", name, "/",
                                  name, "_chr", chr,
                                  "_decreasing.csv") )

}
```

Using `for loop` vs `lapply`. I don't want this to run right now.

```r
class(cleaned_maize$Chromosome) # characters
```

```
## [1] "character"
```

```r
for(chr in as.character(1:10)) {
  final_files(cleaned_maize, chr, "maize")
}

for(chr in as.character(1:10)) {
  final_files(cleaned_teosinte, chr, "teosinte")
}
```

Part II Visualization

SNPs per chromosome

What is the distribution of SNPs on and across chromosomes?

```
# re-join maize and teosinte genotype data
```

Are there more SNP positions in maize or teosinte individuals?

Missing data and amount of heterozygosity

What is the proportion of homozygous and heterozygous sites as well as missing data in each sample and each group? Hints: Create a new column to indicate whether a particular site is homozygous (has the same nucleotide on both chromosomes (i.e., A/A, C/C, G/G, T/T) or heterozygous (otherwise)). Normalize the height of individual bars using one of the ggplot "position adjustments" options.

Your own visualization

Visualize one other feature of the dataset. The choice is up to you!